MLOps CLOUD ENGINEERING LAB

COMPLETE PRACTICAL NOTES

Practicals 1–5

PRACTICAL 1

Building and Training a Machine Learning Model for Telecom Tower Failure Prediction

MLOps Cloud Engineering Lab Manual

MLOps Cloud Engineering Lab – Practical 1b

Title

Building and Training a Machine Learning Model for Telecom Tower Failure Prediction

Aim

To build, train, evaluate, and save a Machine Learning model using Python for predicting telecom tower hardware failures.

Software Required

- Python 3.11 or above
- Python IDLE (or VS Code)
- Internet Connection
- Command Prompt

Required Python Libraries

Open Command Prompt and install the following packages.

```
pip install pandas
pip install scikit-learn
pip install joblib
```

Expected Output

Students should see

```
Successfully installed...
```

Step 1

1.1 Create Project Folder

1.2 Create the following folder: Desktop > MLOPS > mlops prac > prac_1

1.3 Inside prac_1, paste tower_telemetry.csv

Step 2

2.1 Create Python File

2.2 Open Python IDLE

2.3 New File

2.4 Save as train.py inside prac_1

Step 3

3.1 Write Python Program

3.2 Paste the following program:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
import joblib
import json

print("Loading Dataset...")

df = pd.read_csv("tower_telemetry.csv")

X = df[['voltage',
        'temperature_c',
        'signal_drop_rate',
        'cpu_usage',
        'battery_health',
        'network_latency_ms']]

y = df['hardware_failure']

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
    random_state=42
)

print("Training Model...")

model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

model.fit(X_train, y_train)

accuracy = model.score(X_test, y_test)
print("Accuracy :", accuracy)

joblib.dump(model, "telecom_tower_model.pkl")
```

```
metrics = {"accuracy": accuracy}
with open("metrics.json", "w") as f:
    json.dump(metrics, f, indent=4)

print("Training Completed Successfully")
```

3.3 Save the file.

Step 5

5.1 Run the Program

5.2 Open Command Prompt.

5.3 Go to your folder and run:

```
cd "C:\Users\sarve\Desktop\MLOPS\mlops prac\prac_1"

python train.py
```

Expected Output

5.4 Students should see something similar to:

```
Loading Dataset...
Training Model...
Accuracy : 1.0
Training Completed Successfully
```

Step 6

6.1 Check Generated Files

What are these files?

File	Purpose
tower_telemetry.csv	Training Dataset
train.py	ML Training Script
telecom_tower_model.pkl	Trained Machine Learning Model
metrics.json	Model Accuracy Report

Step 7

7.1 Create Prediction Program

7.2 Create another file: predict.py

```
import joblib
import pandas as pd
```

```
model = joblib.load("telecom_tower_model.pkl")

new_data = pd.DataFrame({
    "voltage": [11.2],
    "temperature_c": [55],
    "signal_drop_rate": [0.5],
    "cpu_usage": [88],
    "battery_health": [65],
    "network_latency_ms": [95]
})

prediction = model.predict(new_data)

if prediction[0] == 1:
    print("Hardware Failure Predicted")
else:
    print("Tower is Healthy")
```

7.3 Save the file.

Step 8

8.1 Run Prediction:

```
python predict.py
```

Expected Output

Hardware Failure Predicted

or

Tower is Healthy

depending on the prediction.

PRACTICAL 2

Configuring the CI/CD Pipeline for Telecom Tower Failure Prediction using AWS CodePipeline

Step 1 – Create a GitHub Repository

1.1 Objective: Store the Machine Learning project in a central repository so AWS can access the latest code.

Procedure

1.2 Open GitHub and log in.

1.3 Click New Repository.

1.4 Repository Name: Telecom-MLOps

1.5 Upload the following files: telecom_tower_failure.csv, train.py, requirements.txt, buildspec.yml, README.md

1.6 Click Commit Changes.

Step 2 – Create an Amazon S3 Bucket

2.1 Objective: Store the trained Machine Learning model (model.pkl) in cloud storage.

Procedure

2.2 Login to AWS Console.

2.3 Search for Amazon S3.

2.4 Click Create Bucket.

2.5 Bucket Name: telecom-mlops-model-storage

2.6 Leave all settings as default.

2.7 Click Create Bucket.

Step 3 – Create an IAM Role

3.1 Objective: Give AWS CodeBuild permission to access Amazon S3.

Procedure

3.2 Open IAM.

3.3 Click Roles.

3.4 Click Create Role.

3.5 Select: Trusted Entity – AWS Service; Use Case – CodeBuild

3.6 Attach the following policies: AmazonS3FullAccess, AWSCodeBuildDeveloperAccess, CloudWatchLogsFullAccess

3.7 Role Name: CodeBuildTelecomRole

3.8 Click Create Role.

Step 4 – Create the buildspec.yml File

4.1 Objective: Tell AWS exactly how to build and train the model.

Procedure

4.2 Create a file named: buildspec.yml

4.3 Paste the following code:

```
version: 0.2

phases:
  install:
    runtime-versions:
      python: 3.11
    commands:
      - pip install -r requirements.txt
  build:
    commands:
      - python train.py

artifacts:
  files:
    - model.pkl
```

4.4 Save the file and push it to GitHub.

"buildspec.yml is like an instruction manual for AWS. It tells AWS which libraries to install, which Python file to execute, and which output file (model.pkl) should be saved."

Step 5 – Create an AWS CodeBuild Project

5.1 Objective: Configure AWS to automatically train the Random Forest model.

Procedure

5.2 Open AWS CodeBuild.

5.3 Click Create Build Project.

5.4 Project Name: TelecomModelTraining

5.5 Source Provider: GitHub

5.6 Connect your Telecom-MLOps repository.

5.7 Environment: Operating System – Ubuntu; Runtime – Managed Image; Language – Python

5.8 Service Role: CodeBuildTelecomRole

5.9 Build Specification: Use buildspec.yml

5.10 Click Create Build Project.

Step 6 – Test the Build Process

6.1 Objective: Verify that AWS can successfully train the model.

Procedure

6.2 Open the CodeBuild Project.

6.3 Click Start Build.

6.4 AWS performs the following steps automatically:

```
Download Source Code

↓

Install Python Libraries

↓

Read telecom_tower_failure.csv

↓

Train Random Forest Model

↓

Generate model.pkl

↓

Build Successful
```

6.5 Open Build History > Latest Build > View Logs.

Step 7 – Create an AWS CodePipeline

7.1 Objective: Automate the complete Machine Learning workflow.

Procedure

7.2 Open AWS CodePipeline.

7.3 Click Create Pipeline.

7.4 Pipeline Name: TelecomMLOpsPipeline

7.5 Source Stage: Provider – GitHub Version 2; Repository – Telecom-MLOps; Branch – main

7.6 Build Stage: Provider – AWS CodeBuild; Project – TelecomModelTraining

7.7 Deployment Stage: Skip Deployment

7.8 Click Create Pipeline.

Step 8 – Trigger Automatic Training

8.1 Objective: Verify that the pipeline automatically retrains the model.

Procedure

8.2 Open train.py.

8.3 Change:

```
n_estimators = 100

to

n_estimators = 150
```

8.4 Save the file.

8.5 Commit the changes.

8.6 Push the updated code to GitHub.

Step 9 – Observe Automatic Pipeline Execution

9.1 After pushing the code, AWS automatically starts the pipeline.

9.2 The execution flow is:

```
Developer Updates train.py

↓

Push to GitHub

↓

AWS CodePipeline Detects Change

↓

AWS CodeBuild Starts

↓

Install Libraries

↓
```

```
Execute train.py

↓

Train Random Forest Model

↓

Generate model.pkl

↓

Upload model.pkl to Amazon S3
```

Step 10 – Verify the Output

10.1 Open Amazon S3.

10.2 Open the bucket: telecom-mlops-model-storage

10.3 Verify that the latest model.pkl file is present.

Expected Output

- The updated code is pushed to GitHub.
- AWS CodePipeline automatically detects the change.
- AWS CodeBuild retrains the Random Forest model.
- A new model.pkl file is generated.
- The trained model is uploaded to Amazon S3.
- The entire process is completed without any manual retraining.

PRACTICAL 3A

Amazon S3 — Creating a Bucket, Uploading Dataset and Model

MLOps Cloud Engineering Lab – Practical 3

Step 9: Create an Amazon S3 Bucket

Aim

To create an Amazon S3 bucket for storing datasets and trained machine learning models.

What is Amazon S3?

Amazon S3 (Simple Storage Service) is a cloud storage service provided by AWS. Instead of storing files on your local computer, organizations store datasets, trained models, reports, logs, and backups securely in Amazon S3.

Real-World Example

Think of Amazon S3 as Google Drive for an organization. Instead of:

```
C:\Users\Sarvesh\Desktop\MLOPS
```

we store the files in Amazon S3 where everyone in the organization can access them (subject to permissions).

Architecture

```
Local Computer

↓

tower_telemetry.csv

telecom_tower_model.pkl

↓

Amazon S3 Bucket
```

Step 9.1 – Login to AWS Console

9.1.1 Open your web browser.

9.1.2 Go to: https://console.aws.amazon.com/

9.1.3 Login using your IAM User.

Step 9.2 – Search for Amazon S3

9.2.1 In the AWS Search Bar, type: S3

9.2.2 Click: Amazon S3

Step 9.3 – Create Bucket

9.3.1 Click: Create Bucket

Step 9.4 – Enter Bucket Details

9.4.1 Enter the bucket details:

Field	Value
Bucket Name	telecom-mlops-bucket-yourname
AWS Region	us-east-1 (or your preferred region)

9.4.2 Example: telecom-mlops-bucket-sarvesh

Note: Bucket names must be globally unique. If the name is already taken, add your initials or roll number.

Step 9.5 – Keep Default Settings

9.5.1 Leave all settings as default.

9.5.2 Do NOT change: Bucket Versioning, Default Encryption, Object Ownership, Lifecycle Rules

9.5.3 Scroll down and click: Create Bucket

Expected Output

The bucket appears in the S3 Bucket List (e.g., telecom-mlops-bucket-sarvesh).

Step 10 – Upload Dataset

10.1 Open your bucket.

10.2 Click: Upload

Upload Screen

10.3 Click: Add Files

10.4 Select: tower_telemetry.csv

10.5 Click: Upload

Expected Output

The uploaded dataset (tower_telemetry.csv) should appear inside the bucket.

Step 11 – Upload Machine Learning Model

11.1 Again, click: Upload

11.2 Choose: telecom_tower_model.pkl

11.3 Click: Upload

Expected Output

Your bucket now contains tower_telemetry.csv and telecom_tower_model.pkl.

Step 12 – Create Folder Structure

12.1 Inside the bucket, click: Create Folder

12.2 Create folder: data

12.3 Create another folder: models

12.4 Your bucket should look like:

```
telecom-mlops-bucket-sarvesh
│
├── data
│   └── tower_telemetry.csv
│
└── models
    └── telecom_tower_model.pkl
```

Why Create Separate Folders?

Folder	Purpose
data	Stores datasets
models	Stores trained ML models
logs (later)	Stores build logs
metrics (later)	Stores evaluation reports

This organization makes the bucket easier to manage as the project grows.

Step 13 – Verify Upload

13.1 Open: data

13.2 Verify: tower_telemetry.csv

13.3 Now open: models

13.4 Verify: telecom_tower_model.pkl

13.5 Both files should be visible.

Step 14 – Understanding the MLOps Workflow

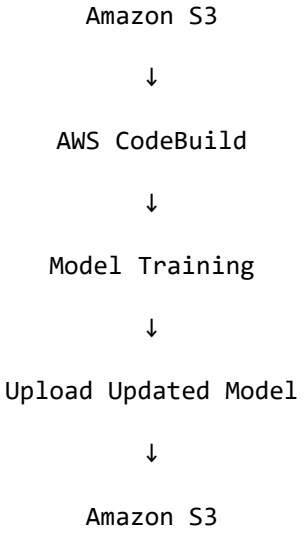
```
Local Computer

↓

tower_telemetry.csv

telecom_tower_model.pkl

↓
```


Amazon S3
↓
AWS CodeBuild
↓
Model Training
↓
Upload Updated Model
↓
Amazon S3

PRACTICAL 3B

Load Testing a Telecom Tower Failure Prediction API using Apache JMeter

Aim

To perform load testing on a Machine Learning API that predicts telecom tower hardware failures and analyze its performance under multiple concurrent users.

Case Study

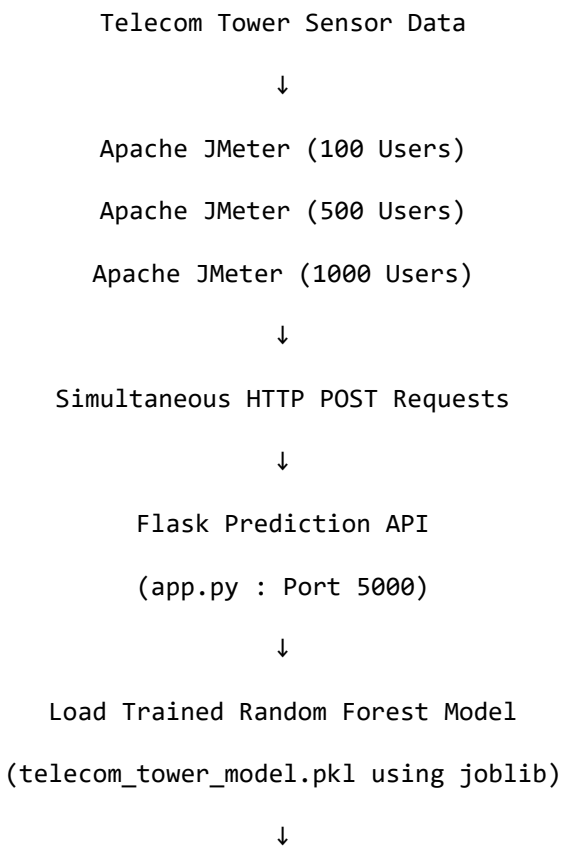
A telecom company has deployed an AI model that predicts whether a mobile tower is likely to fail. Thousands of IoT sensors send requests every second. The company wants to know:

- Can the server handle 100 users simultaneously?
- What happens if 1,000 users send requests?
- Does the response become slow?
- Does the server crash?

This is where Load Testing is performed.

Software Required

- Python
- Flask
- JMeter (Apache JMeter)
- Trained Random Forest Model (telecom_tower_model.pkl)



Random Forest Classifier

↓

Predict Hardware Failure

↓

JSON Response Returned

{"prediction":0} or {"prediction":1}

Step 1 – Install Flask

1.1 Open Command Prompt and run:

```
pip install flask
```

Step 2 – Create Prediction API

2.1 Create: app.py

2.2 Write the following code:

```
from flask import Flask, request, jsonify
import pandas as pd
import joblib

app = Flask(__name__)

model = joblib.load("telecom_tower_model.pkl")

@app.route("/")
def home():
    return "Telecom Tower Prediction API is Running Successfully!"

@app.route("/predict", methods=["POST"])
def predict():
    data = request.get_json()
    df = pd.DataFrame([data])
    prediction = model.predict(df)
    return jsonify({
        "prediction": int(prediction[0])
    })

app.run(debug=True)
```

Step 3 – Run the Flask API

3.1 Open a new Command Prompt window in the project folder (the folder containing app.py and telecom_tower_model.pkl).

3.2 Run:

```
python app.py
```

Expected Output

```
 * Running on http://127.0.0.1:5000/ (Press CTRL+C to quit)
```

Leave this Command Prompt window open — it keeps the Flask server running so it can respond to requests from the browser, Postman, and later from Apache JMeter.

Step 4 – Test API

4.1 Open browser: http://127.0.0.1:5000

4.2 Or test using Postman.

Step 5 – Install Apache JMeter

5.1 Download Apache JMeter from its official website and extract the ZIP file. Launch jmeter.bat (Windows).

Step 6 – Create Test Plan

6.1 Open JMeter.

Test Plan

↓

Thread Group

Configure

Property	Value
Number of Users	100
Ramp-Up Period	10 Seconds
Loop Count	5

Meaning

100 users will gradually access the application over 10 seconds.

Step 7 – Add HTTP Request

Thread Group

↓

Add

↓

Sampler

↓

HTTP Request

Configure

Field	Value
Server	localhost
Port	5000
Method	POST
Path	/predict

Step 8 – Add JSON Data

8.1 Body:

```
{
"voltage":11.2,
"temperature_c":55,
"signal_drop_rate":0.52,
"cpu_usage":85,
"battery_health":62,
"network_latency_ms":90
}
```

Step 9 – Add Listener

Thread Group

↓

Add

↓

Listener

↓

Summary Report

9.1 Also add: View Results Tree

Step 10 – Run Load Test

10.1 Click: Start

JMeter starts sending requests.

Expected Output

Metric	Example
Samples	500
Average Response Time	120 ms
Min	60 ms
Max	240 ms
Throughput	85 Requests/sec
Error %	0%

Understanding the Results

Average Response Time

Average time required for one prediction. Example: 120 milliseconds. Lower is better.

Throughput

Number of requests processed every second. Example: 85 Requests/Second. Higher is better.

Error %

Shows failed requests. 0% means all requests succeeded.

Maximum Response Time

Highest response time observed. Example: 240 ms.

PRACTICAL 4

Installing Git and Uploading Your MLOps Project to GitHub

Aim

To install Git on the local system, create a GitHub repository, and upload the MLOps project for version control.

Step 1: Download Git

1.1 Open your browser and visit: Git Downloads (https://git-scm.com/downloads)

1.2 Click Download for Windows.

Step 2: Install Git

2.1 Double-click the downloaded installer.

2.2 Click Next for all installation screens (keep the default settings).

2.3 Click Install.

2.4 Click Finish.

Step 3: Verify Installation

3.1 Open Command Prompt and type:

```
git --version
```

If Git is installed correctly, you will see something similar to:

```
git version 2.49.0
```

Step 4: Create a GitHub Account

4.1 Go to: GitHub (https://github.com)

4.2 Click Sign Up.

4.3 Complete: Email Address, Password, Username

4.4 Verify your email.

Step 5: Create a New Repository

5.1 After logging in, click the + icon (top-right).

5.2 Select New repository.

5.3 Fill in:

Field	Value
Repository Name	Telecom-MLops

Description	MLOps Practical
Visibility	Public (or Private if preferred)
Initialize README	Leave unchecked

5.4 Click Create Repository.

Step 6: Open Your Project Folder

6.1 Suppose your project is located at: C:\Users\sarve\Desktop\MLOPS\mlops prac\prac_1

6.2 Open Command Prompt.

6.3 Navigate to the folder:

```
cd "C:\Users\sarve\Desktop\MLOPS\mlops prac\prac_1"
```

Step 7: Initialize Git

7.1 Type:

```
git init
```

Output:

```
Initialized empty Git repository
```

A hidden .git folder is created to track all changes.

Step 8: Configure Git (First Time Only)

8.1 Replace with your own name and GitHub email:

```
git config --global user.name "Your Name"
git config --global user.email "your_email@gmail.com"
```

Verify:

```
git config --list
```

Step 9: Check Project Status

9.1 Type:

```
git status
```

Output:

```
Untracked files:
train.py
```

```
app.py
tower_telemetry.csv
metrics.json
telecom_tower_model.pkl
```

9.2 These files are not yet being tracked.

Step 10: Add All Files

10.1 Type:

```
git add .
```

10.2 Now check:

```
git status
```

You should see: Changes to be committed

Step 11: Commit Files

11.1 Type:

```
git commit -m "Initial MLOps Project"
```

Example output:

```
[master] Initial MLOps Project
5 files changed
```

Git has now created the first version (commit) of your project.

Step 12: Copy the Repository URL

12.1 On your GitHub repository page, click the green Code button.

12.2 Copy the HTTPS URL, for example: https://github.com/yourusername/Telecom-MLops.git

Step 13: Connect Local Project to GitHub

13.1 In Command Prompt:

```
git remote add origin https://github.com/yourusername/Telecom-MLops.git
```

Verify:

```
git remote -v
```

Step 14: Push Your Project

14.1 If your default branch is main:

```
git branch -M main
```

14.2 Now push:

```
git push -u origin main
```

GitHub may ask you to authenticate. After a successful push, you'll see output similar to:

```
Branch 'main' set up to track 'origin/main'
```

Step 15: Refresh GitHub

15.1 Open your repository in the browser. You should now see:

```
Telecom-MLops
├── app.py
├── train.py
├── tower_telemetry.csv
├── telecom_tower_model.pkl
├── metrics.json
└── README.md (if added later)
```

Step 16: Make a Change

16.1 Open train.py.

16.2 Add:

```
print("Training Started...")
```

16.3 Save the file.

Step 17: Upload the New Version

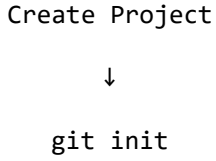
17.1 Run the following commands:

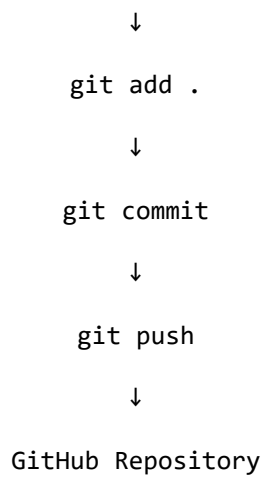
```
git add train.py
git commit -m "Updated training message"
git push
```

17.2 Refresh GitHub.

The updated version of train.py is now stored in the repository.

Git Workflow




git add .
git commit
git push
GitHub Repository

PRACTICAL 5

Monitoring Model Drift Using MLflow and Evidently

Title

Monitoring Model Drift Using MLflow and Evidently

Aim

To monitor the performance of a trained machine learning model on new production data and identify model/data drift using MLflow and Evidently.

Theory

In a traditional machine learning project, a model is trained using historical data and then deployed to make predictions on new data. However, the characteristics of real-world data can change over time. As a result, a model that initially performs well may gradually become less accurate. This phenomenon is known as model drift.

Model drift refers to a situation where the performance or behavior of a machine learning model changes after deployment because the real-world environment or incoming data has changed. Drift can occur due to changes in customer behavior, environmental conditions, business processes, sensor readings, or other external factors.

For example, consider a machine learning model developed to predict hardware failures in telecom towers. The model may be trained using historical values of voltage, temperature, and signal drop rate. If environmental conditions change and the new tower data contains significantly higher temperatures and different voltage patterns, the model may no longer make accurate predictions.

Tools Required

- Python / Python IDLE
- Existing telecom_tower_model.pkl
- Existing tower_telemetry.csv
- MLflow
- Evidently
- Pandas
- Scikit-learn

Step 1: Create Practical 5 Folder

1.1 Create the folder: C:\Users\sarve\Desktop\MLOPS\mlops prac\prac_5

1.2 The folder should initially contain:

```
prac_5
├── telecom_tower_model.pkl
├── tower_telemetry.xlsx
└── new_tower_telemetry.xlsx
```

Step 2: Open Command Prompt

2.1 Go to your practical folder:

```
cd "C:\Users\sarve\Desktop\MLOPS\mlops prac\prac_5"
```

Step 3: Install Required Libraries

3.1 Run:

```
pip install pandas scikit-learn joblib mlflow evidently
```

Step 5 — Check Your Old Model

5.1 Before writing the monitoring program, make sure your .pkl model was trained using the same 8 features in your dataset.

5.2 Your model should have been trained with: Temperature_C, Battery_Voltage, Power_Consumption_W, Signal_Strength_Percent, Fan_Speed_RPM, Humidity_Percent, Traffic_Load, Tower_Age_Years

5.3 ...and the target: Failure_Within_48Hrs

Step 6 — Create monitor_model.py

6.1 Open Python IDLE.

6.2 Click: File > New File

6.3 Save it as: Monitor_model.py

Step 7 — Paste This Code

7.1 Paste the following into monitor_model.py:

```
import pandas as pd
import joblib
import mlflow
from sklearn.metrics import accuracy_score

# ---------------------------------------
# 1. Load the trained Random Forest model
# ---------------------------------------
model = joblib.load("telecom_tower_model.pkl")
print("Model loaded successfully!")

# ---------------------------------------
# 2. Load NEW production data
# ---------------------------------------
new_data = pd.read_excel("new_tower_telemetry.xlsx")
print("New production data loaded successfully!")

# ---------------------------------------
# 3. Select ML features
# ---------------------------------------
```

```
features = [
    "Temperature_C",
    "Battery_Voltage",
    "Power_Consumption_W",
    "Signal_Strength_Percent",
    "Fan_Speed_RPM",
    "Humidity_Percent",
    "Traffic_Load",
    "Tower_Age_Years"
]

X_new = new_data[features]
y_new = new_data["Failure_Within_48Hrs"]

# ---------------------------------------
# 4. Make predictions
# ---------------------------------------
predictions = model.predict(X_new)

# ---------------------------------------
# 5. Calculate accuracy
# ---------------------------------------
accuracy = accuracy_score(
    y_new,
    predictions
)

print("-----------------------------------")
print("MODEL DRIFT MONITORING")
print("-----------------------------------")
print(
    "Production Accuracy:",
    round(accuracy, 2)
)

# ---------------------------------------
# 6. Start MLflow experiment
# ---------------------------------------
mlflow.set_experiment(
    "Telecom_Tower_Model_Monitoring"
)

with mlflow.start_run():
    mlflow.log_param(
        "model",
        "Random Forest"
    )
    mlflow.log_param(
        "n_estimators",
        100
    )
    mlflow.log_metric(
        "production_accuracy",
```

```
        accuracy
    )
    print("Results logged successfully to MLflow")

# ---------------------------------------
# 7. Drift / performance threshold
# ---------------------------------------
threshold = 0.80

if accuracy < threshold:
    print()
    print("WARNING: Model performance has degraded!")
    print("Model retraining is recommended.")
else:
    print()
    print("Model performance is stable.")
```

Step 8 — Run the Program

8.1 Run:

```
python monitor_model.py
```

Expected Output

If everything is correct, you should see:

```
Model loaded successfully!
New production data loaded successfully!
-----------------------------------
MODEL DRIFT MONITORING
-----------------------------------
Production Accuracy: 0.xx
Results logged successfully to MLflow

WARNING: Model performance has degraded!
Model retraining is recommended.
```

Step 9 — Start MLflow

9.1 Open a second Command Prompt window.

9.2 Run:

```
cd "C:\Users\sarve\Desktop\MLOPS\mlops prac\prac_5"

mlflow ui
```

You should see something similar to:

```
Listening at: http://127.0.0.1:5000
```

Step 10 — Open MLflow

10.1 Open Chrome and enter: http://127.0.0.1:5000

10.2 You should see the MLflow dashboard.

10.3 Look for: Telecom_Tower_Model_Monitoring

10.4 Click it. You should see your experiment/run.

Step 11 — Check the MLflow Metrics

11.1 Inside the run, you should find:

Parameters

```
model = Random Forest
n_estimators = 100
```

Metric

```
production_accuracy = 0.xx
```

This demonstrates that MLflow is tracking the model's production performance.

Step 12 — Generate a Data Drift Report Using Evidently

Steps 6–11 use MLflow to log and track the production accuracy of the model, but they do not yet use Evidently even though it is listed under Tools Required. Evidently is used here to visually compare the statistical distribution of the training data (tower_telemetry.xlsx) against the new production data (new_tower_telemetry.xlsx) and to show exactly which of the 8 features have drifted.

12.1 In the same prac_5 folder, create a new file: evidently_report.py

12.2 Paste the following into evidently_report.py:

```
import pandas as pd
from evidently.report import Report
from evidently.metric_preset import DataDriftPreset

reference_data = pd.read_excel("tower_telemetry.xlsx")
current_data = pd.read_excel("new_tower_telemetry.xlsx")

features = [
    "Temperature_C",
    "Battery_Voltage",
    "Power_Consumption_W",
    "Signal_Strength_Percent",
    "Fan_Speed_RPM",
    "Humidity_Percent",
    "Traffic_Load",
```

```
    "Tower_Age_Years"
]

reference_data = reference_data[features]
current_data = current_data[features]

report = Report(metrics=[DataDriftPreset()])
report.run(reference_data=reference_data, current_data=current_data)
report.save_html("data_drift_report.html")

print("Evidently drift report saved as data_drift_report.html")
```

12.3 Save the file.

Step 13 — Run and View the Drift Report

13.1 In Command Prompt (inside prac_5), run:

```
python evidently_report.py
```

13.2 This creates a file named: data_drift_report.html

13.3 Double-click the file (or open it from the browser) to view the report.

Expected Output

The HTML report opens in the browser and shows a Dataset Drift summary (Detected / Not Detected) at the top, followed by a per-feature table for all 8 features with a drift score and a Yes/No drift flag, plus a distribution histogram for each feature comparing the reference and current data.

Features with the largest change in distribution (for example Temperature_C or Power_Consumption_W) are the most likely cause of the drop in production accuracy observed in Step 8, and are good candidates to investigate first before retraining the model.